DESIGN SYSTEM · REFERENCE BUILD

EPB Microsite Style Guide

A mini design system for building presentation-grade internal microsites: one greeter, a set of themed sections, a sticky tab bar, and a library of content blocks that stay legible on a projector.

What this is	Design, layout, navigation and interaction standards — not content.
Who it's for	Anyone recreating this look with an AI coding assistant on another machine.
How to use it	Feed this document into your session as the visual contract, then describe your own content.
Reference build	EPB Own.It Workshop — 5 sections, 27 tabs, 18 block types.

This guide describes a system, not a codebase. Reproduce the rules; write your own markup.

1 Design principles

Five rules explain most decisions in this system. When a choice is unclear, these break the tie.

1.1 Built to be presented, not browsed

Every layout assumes a projector and a live audience. That single constraint drives the rest: generous type, high contrast, no dense grids, nothing that requires a second look. If a table needs sideways scrolling to be read, it is wrong — restructure it or split it into panes.

1.2 One idea per section

A section carries one heading and one block. Alternating background tints separate them, so scrolling reads as a sequence of discrete beats rather than a wall. Long content is chunked behind pill buttons instead of stacked.

1.3 Content is data, not markup

Sections are declared as rows of structured data and rendered by a small set of block types. Nobody edits HTML to change copy. This keeps the design consistent by construction — a contributor cannot invent a new card style by accident.

1.4 Restraint over decoration

Two brand colours, one neutral ramp, three type faces, one accent green used sparingly. Purple carries hierarchy; green marks success and progress; grey does everything else. Add a colour only when meaning demands it.

1.5 Degrade honestly

When something cannot work — an embed that will not frame, a missing image, a tab with no content — say so on screen in plain language. Never leave an empty box that reads as a bug during a live session.

> **The test.** Open any page, stand two metres back, and read it. If you lean in, the design failed, not the reader.

2 Colour

Declare these once as custom properties and never hard-code a hex value anywhere else. Every component below references tokens only.

BRAND

Purple	Purple dark	Purple light	Ink	Green	Lime
`#4B286D`	`#38205A`	`#63398B`	`#2A1A3E`	`#66CC00`	`#8FDD3D`

NEUTRALS AND SURFACES

Tint	Tint 2	Line	Line dark	Grey	Grey light
#F6F2FA	#F0EAF7	#E8E6EC	#D8D5DE	#54505C	#918A9C

```
/* the whole palette — nothing else gets a hex value */
:root{
  --purple:#4B286D; --purple-d:#38205A; --purple-l:#63398B;
  --ink:#2A1A3E;
  --green:#66CC00; --green-d:#4B8500; --lime:#8FDD3D;
  --tint:#F6F2FA; --tint2:#F0EAF7;
  --line:#E8E6EC; --line-d:#D8D5DE;
  --grey:#54505C; --grey-l:#918A9C;
}
```

2.1 Where each colour is allowed

TOKEN	USED FOR	NEVER USED FOR
`--purple`	Headings, big figures, table headers, active states, numerals, badges	Body prose
`--ink`	Body text, hero backgrounds, dark panels	Large fills next to purple — they muddy
`--green` / `--lime`	Active tab underline, progress, success badges, focus rings, one accent per screen	Body text, large areas, anything at small size
`--tint` / `--tint2`	Alternating section backgrounds, card chips, legend panels	Text colour
`--grey`	Secondary prose, captions, table body	Headings
`--line` / `--line-d`	Borders, dividers, dotted leaders	Text or fills

> **Accent discipline.** Green appears at most twice per screen: the active tab marker and one deliberate highlight. It signals "this one" — used everywhere it signals nothing.

2.2 Status colours

Badge pills use a fixed five-state palette. Match a cell's entire value to a keyword and render a pill; anything else stays plain text.

STATE	FILL	TEXT	MEANING
Positive / retained	`#EEF7E0`	`#4B8500`	Unchanged, approved, won
Caution / revised	`#FFF3CD`	`#856404`	Changed, in progress, pending
Negative / removed	`#F8D7DA`	`#721C24`	Deleted, lost, blocked
Neutral / merged	`--tint2`	`--purple`	Consolidated, informational
New	`--purple`	`#fff`	Net addition — strongest state
Unresolved	white, dashed border	`--grey`	Still to validate — looks unfinished on purpose

3 Typography

3.1 Three faces, three jobs

ROLE	STACK	APPLIED TO
Display	`--disp`	All headings, big figures, numerals, card titles
Text	`--text`	Body prose, table cells, captions, buttons
Micro	`--micro`	Eyebrows, table headers, badges, counters — always uppercase with wide tracking

```
--disp:"TELUS SA Display","Helvetica Neue",Arial,sans-serif;
--text:"TELUS SA Text","Helvetica Neue",Arial,sans-serif;
--micro:"TELUS SA Micro","Helvetica Neue",Arial,sans-serif;
```

> **Substituting fonts.** Brand faces are often unavailable. Keep the three-role structure and swap the families — a geometric sans for display, a neutral sans for text, the same neutral for micro. The system depends on the roles, not the typefaces.

3.2 Fluid type scale

Every size is a `clamp()`. No breakpoint-based font sizes anywhere — type scales continuously between phone and projector.

LEVEL	SIZE	WEIGHT	TRACKING
Hero title	`clamp(31px,4.8vw,64px)`	900	`-.028em`
Section head	`clamp(23px,2.7vw,37px)`	750	`-.024em`
Column head	`clamp(24px,2.5vw,34px)`	900	`-.028em`
Big figure	`clamp(26px,3vw,38px)`	900	`-.026em`
Card title	17–20px	750	`-.018em`
Body	15–16px	400	0
Table cell	13.6px	400	0
Table header	10.5px	500	`.11em` uppercase
Micro label	10–11px	500	`.13–.19em` uppercase

3.3 Rules

- **Negative tracking scales with size.** Display type tightens (`-.02` to `-.035em`); body and micro never do.
- **Weight is binary.** 900 for display, 750 for card titles, 400–500 for everything else. No 600s.
- **Measure is capped.** Prose blocks stop at `74ch`; bullet lists at `88ch`. Full-width paragraphs are unreadable at range.
- **Uppercase only in micro.** Never uppercase a heading or body text.
- **Balance short headings** with `text-wrap:balance` so two-line titles split evenly.

4 Layout and spacing

4.1 One gutter token

A single fluid gutter governs every horizontal edge. Content aligns to it without exception — headings, tables, grids and cards all start at the same x-position, which is what makes the page feel engineered rather than assembled.

```
--gut: clamp(20px, 5vw, 72px);
.pad{ padding-left:var(--gut); padding-right:var(--gut) }
```

> **Do not cap content width inside the gutter.** A `max-width` on a grid makes it narrower than the table above it and the misalignment is immediately visible. Let the gutter do the work.

4.2 Vertical rhythm

ELEMENT	SPACING
Section padding	`clamp(50px,6.5vw,86px)` top and bottom
Heading to content	`34px`
Grid gap	`14–20px` — tighter for dense cards, looser for charts
Card padding	`clamp(20px,2.2vw,30px)`
First section clearance	`var(--navh) + clamp(28px,3vw,44px)`

4.3 Alternating tint

Sections alternate white and `--tint` by index, with a hairline border top and bottom on tinted ones. This is the primary device separating one idea from the next.

```
.sect{ padding:clamp(50px,6.5vw,86px) 0 }
.sect.tint{ background:var(--tint);
  border-top:1px solid var(--line); border-bottom:1px solid var(--line) }
```

> **Continuation blocks.** A block with no heading belongs to the section above it: give it that section's tint and drop its top padding, so the two read as one beat rather than two stripes. Re-derive the stripe order from the top whenever sections are reordered.

4.4 Grid defaults

PATTERN	COLUMNS	COLLAPSE POINT
Fixed pair (2×2)	`repeat(2,1fr)`	1 column below 760–900px
Fixed quad	`repeat(4,1fr)`	2 at 1180px, 1 at 640px
Flexible cards	`auto-fit,minmax(280px,1fr)`	Self-collapsing
Asymmetric split	`1.15fr .85fr`	1 column below 1024px

Prefer a **fixed** column count when the item count is known and small (four items should read as a square, not a row of four on a wide screen). Use `auto-fit` only when the count varies.

4.5 Test viewports

Verify at four sizes, in this order of importance:

VIEWPORT	WHY
1366 × 768	The tightest realistic laptop — where things break first
1440 × 900	Most common working size
1920 × 1080	Projector and large display
375 × 812	Phone — must never scroll sideways

5 Site structure and navigation

5.1 The three-level model

Every site in this system has exactly three levels. Resist adding a fourth.

LEVEL	ELEMENT	PURPOSE
1 — Greeter	Full-height landing panel	Sets the occasion. Wordmark, place and date, one question, a scroll cue.
2 — Section picker	Grid of cards	One card per major theme. Card = number, title, one-line blurb, "Open".
3 — Tabbed content	Sticky tab bar + body	Each section opens its own tab bar. Tabs are chapters within that theme.

5.2 The greeter

- **Fits the fold.** Panel height must be ≤ viewport at every test size, with the scroll cue visible. This is the tightest constraint in the system — verify at 1366×768 whenever you add anything to it.
- **Image-in-type.** The wordmark is a photo masked inside the letterforms via an SVG mask, not text over an image. Measure the rendered glyphs and snap the viewBox to them so the word fills the frame regardless of which font actually loaded.
- **Stack:** small logo → wordmark → place and date (micro, uppercase, wide tracking) → one question in display type → chevron.
- Reveal the logo only once it loads, so a broken URL leaves no gap.

5.3 Section cards

Vertical cards on a dark translucent surface: number label, title, blurb, and an "Open →" cue that appears on hover. A coloured rule wipes in across the top on hover. No icons — they compete with the numbers and add nothing.

Below 760px the cards become a stacked list with the number tucked into the corner and the "Open" cue hidden.

5.4 The sticky tab bar

Fixed to the top, transparent over a hero, and switching to an opaque state with a shadow once the page scrolls past it.

PROPERTY	RULE
Sizing	`flex:1 1 auto` — each tab sized to its own label
Overflow	**Wrap to a second row.** Never truncate or ellipsis a label
Wrapped rows	Centred, `flex-grow:0`, tighter padding, hairline top border
Active state	Tinted background plus a 4px accent bar at the bottom edge
Below 900px	Collapses to a drawer: current tab name plus a hamburger

> **The fixed-bar trap.** A wrapping bar changes height, so any hardcoded content clearance breaks the moment it wraps. Measure the bar in JS, publish the height as a custom property, and drive both the first section's top padding and `scroll-margin-top` from it. Re-measure on resize, and again after the container becomes visible — elements measure zero while hidden.

```
// detect the wrap — CSS cannot report it
tabs.forEach(t => { if (Math.abs(t.offsetTop - tabs[0].offsetTop) > 2) wrapped = true; });
row.classList.toggle('wrapped', wrapped);
document.documentElement.style.setProperty('--navh', nav.offsetHeight + 'px');
```

```
.view > section:first-child:not(.hero){
  padding-top: calc(var(--navh, clamp(120px,13vw,180px)) + clamp(28px,3vw,44px)) }
.view section{ scroll-margin-top: calc(var(--navh, 88px) + 12px) }
```

5.5 Tab heroes

Each tab opens with a short hero: eyebrow (`Section 03 · Name`), a one-line title, and a lede sentence, over a tinted image on an ink background. It orients the room before any data appears.

5.6 The persistent scroll button

One floating control, bottom-right, that does double duty: "Back to top" while scrolling, and "Next section →" once the page bottom is reached. On a page too short to scroll it shows "Next section" immediately — otherwise it would never appear at all.

6 The block library

Content is declared as rows — `section` , `tab` , `type` , `title` , `body` — and rendered by a fixed set of block types. A contributor picks a type and fills a body string; they never touch layout. This is what keeps a large site visually consistent.

6.1 Body grammar

SEPARATOR	SPLITS
`|`	Items in a list, or cells in a table row
`::`	Fields within one item

`||`	Rows of a table
A rare glyph, doubled	Groups — panes of a tabbed block

Two conventions matter more than the exact characters: separators must be **characters no author would type naturally**, and the same three must mean the same thing in every block type.

> **Escaping.** The renderer HTML-escapes every value, so write literal `<` and `>` in your data — never pre-escaped entities, which display as visible text. Sanitise separator characters out of imported content before it reaches a body string, or the table silently gains columns.

6.2 The eighteen types

TYPE	BODY	RENDERS AS
`intro`	prose	Heading plus paragraph, capped measure
`note`	prose	Dark gradient callout with a lime badge
`bullets`	`Item|Item`	Plain list, coloured markers
`cards`	`Label::Detail|…`	Flexible card grid
`stats`	`Value::Caption|…`	Big-number tiles, accent left border
`insights`	`Label::Figure::Detail|…`	4-across bento: numbered badge, header across the top, figure left / prose right
`bento`	`Item|…`	2×2 numbered prose cards
`pairs`	`Name::Value|…`	Two-column ranked list, dotted leader, name bold
`steps`	`Title::Detail|…`	Numbered process cards
`expand`	`Label::Summary::Detail|…`	Click-to-expand cards
`split`	two groups	Two columns: numbered cards left, stat tiles right
`table`	`Head|Head||r1|r1`	Data table
`tabbed`	groups of tables	Pill buttons revealing one table
`cardswitch`	groups with subtitles	Cards revealing one table
`swap`	groups of steps	Pill buttons revealing step cards
`image`	caption + URL	Full-width, click-to-expand
`imggrid`	`Caption::URL::Footnote|…`	2×2 chart grid, footnote in both views
`embed`	a URL	Responsive iframe with a new-tab fallback

> **Adding a type.** Only when an existing one genuinely cannot express the content — and never for a one-off. Each new type is a permanent obligation: it must handle empty fields, degrade on mobile, and be documented here. Eighteen is already generous.

6.3 Worked examples

Five bodies showing the grammar in use. Everything else follows these shapes.

```
// cards — a flexible grid
Process Design::Define the requirements for an optimal process.|Managing Risk::Cl\
arify approvals for non-standard terms.|RACI::Eliminate ownership ambiguity.

// stats — value first, caption second
7,412::total contracts in 2025|4,681::contracts so far in 2026

// table — header row, then rows split on ||
Year|Volume|Share||2025|7,412|61%||2026|4,681|39%

// insights — label, figure, then the sentence
Mid-Market Dominance::**53.31%**::The largest share across both years.|Public Grow\
th::14.99% to 20.63%::A noticeable increase in share.

// tabbed — two panes, each holding a whole table
2025::Segment|Total||ENT|2,412||MM|3,855<SEP>2026::Segment|Total||ENT|1,486||MM|2,242
```

NOTE `<SEP>` stands for your chosen group separator — a doubled rare glyph. Wrapping a figure in a marker convention such as `**…**` promotes it to a highlight.

7 Component specifications

7.1 Cards

All card variants share one anatomy: white fill, hairline border, 10–14px radius, and a 3px coloured edge that carries meaning.

VARIANT	EDGE	HOVER
Content card	3px top, neutral → accent	Lift 3px, shadow
Stat tile	3px left, purple	None — not interactive
Bento tile	3px top rule, wipes in	Lift, numeral opacity rises
Expandable	Border tints when open	Chevron rotates 180°

```
.card{ background:#fff; border:1px solid var(--line);
  border-top:3px solid var(--line-d); border-radius:10px; padding:28px;
  transition:border-top-color .2s, transform .2s, box-shadow .2s }
.card:hover{ border-top-color:var(--green); transform:translateY(-3px);
  box-shadow:0 14px 32px rgba(75,40,109,.09) }
```

7.2 Numbered treatments

Three distinct devices — do not mix them within one block.

- **Badge** — small tinted chip, top-left corner, `01` zero-padded. For ranked findings.
- **Ghost numeral** — large display digit at low opacity beside the text, rising on hover. For two to four peer items.
- **Step marker** — solid digit in a tinted square, left of the content. For sequences.

7.3 Tables

The single most fragile component in a presentation context. Three rules keep them readable.

1. **Figures never wrap; prose always does.** Default cells to `nowrap`, then detect prose columns and let those wrap. Judge by the longest cell in the column — over ~28 characters and three or more words — and tag the whole column, never individual cells.
2. **Wide tables wrap their headers.** At five or more columns, allow the header row to break onto two lines and tighten the gutters. The nowrap header row is usually what pins a table too wide.
3. **Sticky header, scrollable shell.** Wrap every table in an `overflow-x:auto` container so a stubborn table scrolls inside its own box rather than pushing the page sideways.

> **Verify hidden panes.** A table inside an unselected pane measures zero. Click through every pane before declaring a page clean — overflow bugs hide there.

EMPHASIS INSIDE TABLES

- Wrap a cell in a marker convention (for example `**value**`) to tint the whole row, add a left marker and bold the figure.
- A cell whose **entire** value matches a status keyword renders as a pill. Partial matches must not — split the keyword out of the prose first.

7.4 Legends for abbreviations

When column headers are codes (`L1` , `L4`), put a legend block above the table: a grid of chips mapping each code to its meaning, styled to match the table header so the association is instant.

> **Keep a legend in code, not content.** If the same mapping appears in two places it will drift. Render it from one array.

8 Interaction and motion

8.1 Motion budget

INTERACTION	DURATION	EASING
Hover lift, colour	200ms	default
Rule wipe, chevron	280–300ms	`ease`
Pane reveal	280–340ms	`ease`
Entrance reveal	900ms	`cubic-bezier(.2,0,0,1)`
Counter, chart draw	1200–1400ms	`cubic-bezier(.3,0,0,1)`

Nothing animates on a loop. Every animation runs once, on entry or on click.

8.2 Reveal on scroll

Sections fade and rise slightly as they enter the viewport, via an IntersectionObserver that unobserves after firing. Staggered by 60–80ms within a group.

> **Two traps that produce blank content.**
> • **Reduced motion** must paint the *end state* immediately, not skip the animation — otherwise counters sit at zero and charts stay empty.
> • **Background tabs suspend animation frames.** Check for a hidden document alongside the reduced-motion query and take the same immediate-paint path, or a section prepared in a background tab renders blank when the user returns.

```
// one gate for both conditions, checked by every animation
const still = () =>
  matchMedia('(prefers-reduced-motion:reduce)').matches || document.hidden;
```

8.3 Progressive disclosure

Three patterns, in increasing weight:

- **Pill buttons** — one pane visible at a time. For alternative cuts of the same data (by year, by region).
- **Selector cards** — same mechanism, larger targets with subtitles. For a small number of substantial choices.
- **Expandable cards** — independent open state, several may be open at once. For depth that is optional. Animate a measured height, keep the pane in layout so it stays measurable, and mirror the state in `aria-expanded`.

8.4 Modals

Two only: a lightbox for images and a document preview for embeds. Both set `role="dialog"`, lock body scroll, move focus to the close control, and restore it on close.

- Set an iframe `src` only when the modal opens — never load a dozen embeds on page load.
- **A caption shown under a thumbnail must also appear in the expanded view.** A definition visible in only one place is the one the room asks about in the other.

8.5 Accessibility floor

- Focus ring: 3px accent, offset, on every interactive element. Never removed.
- Real `<button>` elements for controls; anchors only for navigation.
- `aria-label` on every icon-only control; `aria-expanded` on every toggle; `alt` on every image.
- Meaning never carried by colour alone — badges carry text, active tabs carry a bar.
- Full reduced-motion support with end states painted.

9 Failure states and hard-won constraints

These come from things that actually broke during a live build. They are the most valuable part of this guide.

9.1 Embeds that refuse to frame

Many hosted documents send headers that forbid embedding. The browser renders nothing and the modal looks broken.

Handle it in code. Classify the URL shape before rendering. If it cannot be framed, show a panel explaining why, with a working "Open in a new tab" button — never an empty iframe.

SHAPE	FRAMES?
Document `/preview` paths	Usually yes
File `/preview` paths	Usually yes
Folder or listing views	**Never** — blocked by header
`/edit` paths	No — use `/preview`

Also assume embeds require the viewer to be signed in. Always pair one with a new-tab link.

9.2 Data-shape drift

> **Inserting a column silently breaks everything downstream.** If the renderer reads fixed positions and the source gains a column, every field shifts by one. The visible symptom is unrelated — links stop working, a modal opens blank — and nothing errors.
>
> Guard it three ways: read one column more than you need; detect shape per row and normalise before rendering; and validate the header at load, logging loudly when it does not match.

9.3 Empty and partial states

SITUATION	REQUIRED BEHAVIOUR
Tab with no content	Friendly panel naming where content comes from
Missing image URL	Placeholder with the caption and what is needed
Empty filter result	Explain the filter and how to populate it
Broken logo or hero	Stay hidden until loaded — leave no gap
Optional field absent	Collapse the slot; never render an empty label

9.4 Numbers you publish

- **Reconcile before transcribing.** Check that rows sum to their stated totals. Source documents contain typos.
- **Never invent a total that does not add up.** If the parts sum to less than the stated whole, label it "listed total" and note the gap.
- **Recompute derived counts** when a percentage changes. A detail sentence quoting a count from an old percentage becomes wrong silently.
- **Sanity-check averages** against their own distributions. An average that contradicts its tiers is a data error worth raising before presenting.

9.5 Encoding consistency

Pick one representation for special characters — escaped sequences or literal glyphs — and use it everywhere. A file mixing both looks fine but a find-and-replace on one form silently misses the other.

10 Build checklist

10.1 Getting started with an AI assistant

1. Share this document and state that it is the visual contract.
2. Declare the tokens from §2 and §3 first, before any component.
3. Build the three-level shell from §5 — greeter, cards, tab bar — and verify the greeter fits the fold at 1366×768.
4. Implement only the block types you need. Start with `intro`, `cards`, `stats`, `table`, `note`.
5. Keep content in data rows, separate from the renderer, from the first commit.
6. Add richer types as content demands, following §6.2 exactly.

10.2 Before every review

LAYOUT

- Tables fit, no sideways scroll
- Every pane of every toggle checked
- No cell text clipped
- Alternating tint still correct
- Grids align to the gutter
- No page-level horizontal scroll

NAVIGATION

- Tab bar wraps rather than truncates
- Clearance follows the measured bar height
- No heading hidden under the fixed bar
- Drawer works below 900px

CONTENT

- No literal HTML entities visible
- Table rows all have equal column counts
- Totals reconcile
- Derived counts match their percentages
- Every embed has a new-tab fallback

INTERACTION

- Every toggle reveals its pane
- Expandables open and close
- Modals trap and restore focus
- Reduced motion paints end states
- No console errors

10.3 Automate the fragile checks

Two scripts pay for themselves immediately:

- **A function inventory.** Compare every declared function against every handler referenced in markup. A syntax check cannot catch a deleted function — missing code is valid until something calls it, and the callers are strings in HTML. Bulk edits destroy functions; this is how you find out before a user does.
- **A layout sweep.** Walk every tab and every hidden pane in the browser, measuring table overflow, clipped cells, literal entities and page scroll. Manual checking misses hidden panes every time.

> **Verify, then claim.** Run the checks and read the output before saying something works. Screenshots may be unavailable in headless environments — measure geometry instead of trusting a visual glance.

10.4 What makes this system work

Not the purple, and not the fonts. It is three structural commitments:

1. **Content as data** — so consistency is automatic rather than policed.
2. **A fixed block vocabulary** — so every page is assembled from known-good pieces.
3. **Honest failure states** — so nothing looks broken in front of an audience.

Swap the palette and the typefaces for your own brand and the system still holds. Abandon those three commitments and it stops working, whatever colours you use.
